



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

並行程序設計實驗報告

GPU 編程實驗

姓名：呂昊遠

學號：2410668

專業：計算機科學與技術

指導教師：王剛

2026 年 6 月 20 日

目录

第一部分	AMD 平台学习	1
第二部分	GPU 编程实验	17
一、	实验背景	17
二、	实验原理	17
(一)	GPU 与 CUDA 编程模型	17
(二)	算法回顾	18
(三)	批量化与簇重叠率	18
三、	程序设计与实现	19
(一)	通用头文件与共享设备 kernel	19
(二)	暴力检索 (gpu_flat.cuh)	20
(三)	IVF (gpu_ivf.cuh)	20
(四)	IVF-PQ: (gpu_ivf_pq.cuh)	21
(五)	批量测试框架	21
四、	实验结果分析	21
(一)	数据总览	21
(二)	暴力检索: GPU+CPU 与 GPU+GPU	22
(三)	IVF 相关参数分析	23
(四)	IVF-PQ	26
(五)	参数权衡	27
(六)	问题探讨	27
五、	总结与体会	28
六、	附录: 代码仓库链接	29
七、	附录: AI 使用说明	29

第一部分 AMD 平台学习

Lab1 : Vector Addition

Exercise: Thread Index Calculation

Fill in the global index for each thread:

blockIdx.x	blockDim.x	threadIdx.x	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

通过索引公式 $globalIdx = blockIdx.x * blockDim.x + threadIdx.x$ 直接计算即可。

Exercise1 为 GPU 基础信息的填空，此处跳过。

Exercise 2: Implement the Kernel

Complete the kernel in based on the structure above. `vector_add_kernel.hip`

补全的代码如下：

```

1  int idx = blockIdx.x * blockDim.x + threadIdx.x; // 计算全局索引
2  if (idx < N) {                                // 边界检查
3      C[idx] = A[idx] + B[idx];                  // 执行加法
4  }
```

Exercise 3: Launch Configuration Calculation

Given elements, complete the table: $N = 1000$

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\text{ceil}(1000/64)$	16	1024	24
128	$\text{ceil}(1000/128)$	8	1024	24
256	$\text{ceil}(1000/256)$	4	1024	24
512	$\text{ceil}(1000/512)$	2	1024	24

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer: 在此特定例子中 ($N = 1000$)，所有这四种 Block Size 的利用率都是完全一样的 ($1000/1024 = 97.66\%$)。然而在实际情况中，过小的 Block Size (如 16) 可能会因为无法填满一个完整的 Wavefront (RDNA 为 32) 而导致硬件利用率低下；过大的 Block Size 则可能受到寄存器或共享内存的限制，降低整体 Occupancy。

计算公式: $Grid = \lceil N / BlockSize \rceil$

Exercise 4: Analyze Your Results

Based on the multi-trial results above:

1. Which block size achieved the **lowest mean execution time**? **512 (0.1402)**
2. Which block size achieved the **highest memory bandwidth**? **512**
3. Are the differences between block sizes **statistically significant** (look at error bars)? **不是很显著，相对误差最大的blocksize = 32也仅仅只有0.02ms的浮动**
4. Why does Vector Add performance depend more on bandwidth than on block size?

Your answer: **因为向量加法的算术强度很低，每进行一次浮点加法运算，就需要读入 2 个浮点数并写出 1 个浮点数，即读写 12 bytes 内存。只要 Block Size 合理，GPU 就能迅速达到内存总线的带宽瓶颈。此时，性能瓶颈在显存带宽，而非计算单元。**

Exercise 5: Wavefront Analysis

Question 1: A kernel is launched with block size 100. How many wavefronts does each block require? How many lanes are wasted?

- Wavefronts per block: **4** (hint: $\lceil 100/32 \rceil$)
- Wasted lanes: **28** (hint: wavefronts $\times$ 32 - 100)

Question 2: If the same kernel were compiled with `-mwavefrontsize64`, how would the answers change?

- Wavefronts per block (wave64): **2** (hint: $\lceil 100/64 \rceil$)
- Wasted lanes (wave64): **28**

Question 3: What happens when threads within a wavefront take different branches (if-else)?

Your answer: **这会导致线程发散。Wavefront 中的所有线程必须同步执行相同的指令。如果出现 if-else 分支，硬件会依次串行执行两个分支，不需要执行当前分支的线程将被暂时屏蔽，这会成倍增加指令周期，降低计算效率。**

Exercise 6: Occupancy Calculation

For Radeon 8060S (RDNA 3.5): 20 CUs, Warp Size = 32, Max Blocks per CU = 64, Max Threads per Block = 1024

To calculate occupancy, you need to know the max wavefronts per CU (check `rocminfo` output).

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	<i>max threads</i>
64	2	32	64	<i>max threads / wavefronts</i>
128	4	16	64	<i>max threads / wavefronts</i>
256	8	8	64	<i>max threads / wavefronts</i>
512	16	4	64	<i>max threads / wavefronts</i>
1024	32	2	64	<i>max threads / wavefronts</i>



Exercise 7: Analyze Sub-Wavefront Results

Based on the multi-trial results above:

- Mean time for block size 8 (sub-wave): **0.1673** ms
- Mean time for block size 16 (sub-wave): **0.1483** ms
- Mean time for block size 32 (wave-aligned): **0.1424** ms
- Mean time for block size 256 (large): **0.1385** ms

Question 1: Is there a noticeable performance penalty for sub-wavefront block sizes (8, 16) compared to wave-aligned sizes (32, 64, 128)?

Your answer: **Yes.** 在 RDNA 3.5 上硬件调度的最小单位是 32 个线程。如果 Block Size 设为 8 或 16，硬件依然会分配一个完整的 Wave32 来执行它，导致一半以上的计算资源和内存请求槽位白白闲置，性能显著下降。

Question 2: On this RDNA 3.5 GPU, the wavefront size is 32. If you were on an MI100 (wavefront = 64), which block sizes would become sub-wavefront?

Your answer: **小于 64 的尺寸，即 8, 16, 32。**

本实验带领我们学习了基于 HIP 的 AMD GPU 编程的核心模型与底层硬件映射关系，理解了 GPU Kernel 的软硬件对应关系。Grid 对应整个 GPU 任务，Block 对应 Compute Unit (CU)，而 Thread 则运行在具体的 SIMD Vector Lane 上。

还讲解了 1D 线性内存的线程分配公式 ($idx = blockIdx.x * blockDim.x + threadIdx.x$)，以及如何使用边界检查避免内存越界。以及 Launch Configuration (启动配置) 部分，讲解了如何通过数据量 N 和 Block Size 计算 Grid Size，并分析了部分线程闲置带来的 Utilization (利用率) 折损问题。

此外，还深入学习了 RDNA 3.5 架构下 Wave32 的硬件执行特性。非 32 倍数的 Block Size 会导致部分 Vector Lane 闲置；而 Sub-wavefront (小于 32) 的 Block Size 会遭受严重的性能惩罚。以及分支发散 (Thread Divergence) 会导致指令串行化的这一效果。

Lab2 : Matrix Transpose

Exercise 1: 2D Indexing

Given `blockDim = (32, 32)`, calculate the global indices:

<code>blockIdx</code>	<code>threadIdx</code>	Global (<code>idx, idy</code>)
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(32, 0)
(2, 3)	(15, 20)	(79, 116)

Question: For a matrix with `rows=100, cols=200`, how many blocks are needed?

Your calculation: **28**

直接代入公式即可: $idx = blockIdx.x * blockDim.x + threadIdx.x$ 和 $idy = blockIdx.y * blockDim.y + threadIdx.y$

Exercise 2: Understand the Index Transformation

For a 4x6 matrix (`rows=4, cols=6`), fill in the table:

Thread (<code>idx, idy</code>)	Input Index	Output Index
(0, 0)	$0*6+0 = 0$	$0*4+0 = 0$
(1, 0)	1	4
(0, 1)	6	1
(3, 2)	15	14

Question: Why is the write index formula `idx * rows + idy` instead of `idx * cols + idy`?

Your answer: 因为输出矩阵是输入矩阵的转置，其维度变成了 (`cols × rows`)。在输出矩阵中，每一行有 `rows` 个元素，因此计算线性索引时，跨越一行的步长 (Stride) 应该是 `rows` 而不是 `cols`。

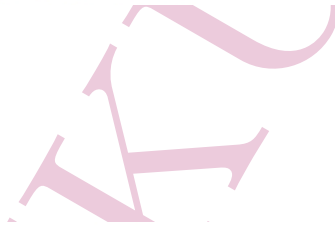
Exercise 3: Performance Analysis

Record execution times:

Test	Dimensions	Elements	Time
1	16×16	256	0.0075
2	128×32	4,096	0.0104
3	1×1024	1,024	0.0085
4	1001×2001	2,003,001	0.2366

Question: The large test has ~500x more elements than test 2. Is the time 500x longer? Why or why not?

Your answer: 通常不会。对于小矩阵 (Test 1、2)，GPU 的计算单元没有被填满，执行时间主要被 Kernel 的启动开销 (Launch Overhead) 所主导。而对于大矩阵 (Test 4)，真实的数据读写耗时才会显现，且由于朴素转置存在大量的非合并内存写入 (Non-coalesced writes)，其平均每个元素的耗时会显著高于预期。



Exercise 4: Coalescing Analysis

For a 1024×1024 matrix with 32×32 thread blocks (one wave32 = one row of the block):

Question 1: When reading `input[idy * cols + idx]`, are consecutive threads (in `threadIdx.x`) reading consecutive addresses?

Your answer: 连续的线程拥有连续的 `idx`，而 `idx` 在输入公式中是内层维度 (步长为1)，因此它们读取的是连续的内存地址，实现了合并读取。

Question 2: When writing `output[idx * rows + idy]`, what is the stride between addresses written by consecutive threads?

Your answer: *Stride* 为 *rows* (即 1024)。连续的线程 `idx` 相差 1，而写入公式中 `idx` 乘以了 *rows*，导致内存地址跳跃。

Question 3: If each memory transaction fetches 128 bytes (32 floats), how many transactions are needed for 32 threads (one wavefront) to write non-coalesced?

Your answer: 32 次。因为每次写入的地址都相隔了 1024 个浮点数 (4096 bytes)，远超过一个 *transaction* 的 128 bytes 范围，所以每个线程都需要发起一次独立的内存事务。

Exercise 5: Tiled Transpose Analysis

Question 1: Why do we add `+1` to the shared memory declaration `( tile[BLOCK_SIZE][BLOCK_SIZE + 1] )`?

Your answer: 为了避免 *Shared Memory Bank Conflicts*。GPU 的共享内存通常被划分为 32 个 Banks，如果步长是 32 (`BLOCK_SIZE`)，同一列的线程将访问同一个 Bank 导致串行等待。增加 1 后步长变为 33，使得同一列的元素错开分布到了不同的 Bank 中。

Question 2: In the write step, why do we swap `blockIdx.x` and `blockIdx.y`?

Your answer: 为了确保写入全局内存时依然是连续的（合并写入）。我们将转置的动作在速度极快的共享内存中完成，然后在写回全局内存时，每个线程块负责向输出矩阵中属于自己的转置后位置，按行连续地写回数据。

Question 3: How much shared memory does each block use for `BLOCK_SIZE=32`?

Your calculation: 4224 bytes

$$32 \times (32 + 1) \times 4 \text{ bytes (float 的大小)} = 4224 \text{ bytes.}$$

本实验主要介绍了 GPU 编程中的 2D 线程索引与内存合并访问 (Memory Coalescing)。

在 2D 网格与内存布局这部分，学习了如何使用 `blockIdx.x/y` 和 `threadIdx.x/y` 处理二维数据，并掌握了 C/C++ 中行优先 (Row-Major) 存储导致的线性索引计算公式 ($\text{idx} = y * \text{cols} + x$)。以及 GPU 内存事务以 Wavefront 为单位的特点，当连续的线程访问连续的内存地址时，带宽利用率最高。

在朴素的矩阵转置算法中，读取行（合并访问）会导致写入列（步长巨大的非合并访问），从而引起严重的性能瓶颈。通过在每个 Block 内引入 Shared Memory 作为一个缓冲切片，实现了合并读取、在共享内存中转置、最后合并写回的优化策略。此外，通过给共享内存维度加一 (Padding)，也成功避免了 Bank Conflicts 的问题。

Lab3 : Histogram - Atomic Operations

Exercise 1: Analyze the Problem

Question 1: How many times is each input element read from global memory?

Your answer: 每个输入元素被读取 `num_bins` 次 (每个 block 各扫一遍完整数组)

Question 2: If `num_bins = 256` and `N = 10M`, what is the total global memory traffic?

Your calculation: $10M \times 256 = 2,560,000,000$ 次读取 ($\approx 10.24 \text{ GB}$)

Question 3: Why is this approach inefficient?

Your answer: 内存流量为 $O(N \times \text{num_bins})$ ，绝大多数读取无效 (每个 block 只考虑一个值)，bins 越多浪费越严重

Exercise 2: Understand the Optimization

Question 1: Why is atomicAdd on shared memory faster than on global memory?

Your answer: **Shared Memory (LDS)** 是片上存储, 延迟约 20 cycles, 而 **Global Memory** 约 400 cycles; 且竞争只在 block 内部 (~256线程), 而非全 GPU, 冲突概率大幅降低

Question 2: In the merge step, how many global atomicAdd operations occur per bin?

Your answer: **merge 阶段每个 bin 共执行 gridDim.x 次 global atomicAdd (每个 block 1 次)**

Question 3: What is the purpose of the grid-stride loop pattern `for (int i = idx; i < N; i += totalthread) ?`

Your answer: **Grid-stride loop** 让固定数量线程处理任意大小数组, 每个线程负责多个元素, 避免启动过多 block, 提升 occupancy 和 cache 利用率



Exercise 3: Record Results

Fill in the execution times:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	2.3634 ms	22.5639 ms
Naive GPU	23.0717 ms	2562.8900 ms
Optimized GPU	0.4581 ms	4.4466 ms

Calculate speedups:

- Optimized vs Serial: **5.1 x**
- Optimized vs Naive: **576 x**

Exercise 4: Memory Efficiency

Question: Calculate the memory traffic reduction ratio:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{N \times \text{num_bins}}{N} = ?$$

Your answer: = **num_bins**, 即优化方案全局内存读取量仅为朴素方案的 1/num_bins

Exercise 5: Contention Scenarios

Scenario 1: All input values are the same (e.g., all zeros)

- What happens to performance? **性能最差**
- Why? **所有线程争同一个 bin, shared memory atomic 完全串行化**

Scenario 2: Input values are uniformly distributed across all bins

- Expected contention level: **竞争最低**
- Why? **更新均摊到所有 bins, 每个 bin 碰撞极少, 性能最优**

本实验实现了用 GPU 并行计算整数数组的直方图，主要学习了两个核心概念：一是使用 atomicAdd 原子操作避免多线程并发写入同一 bin 时产生的 Race Condition；二是利用 Shared Memory 私有化（每个 block 维护局部直方图，最后再合并到全局），将 atomic 竞争范围从全局缩小到 block 内部，从而大幅降低延迟和冲突。实验通过实际的运行测试，分别对比了朴素方案（每个 block 扫全数组）与优化方案的性能差异。

Lab4 : Parallel Reduction

Exercise 1: Understand Tree Reduction

Question 1: For an array of 1024 elements with block size 256, how many reduction steps occur within each block?

Your answer: **8 步。每步 stride 减半: $128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$, 共 $\log_2(256) = 8$ 次迭代。**

Question 2: After block-level reduction, how many partial sums remain?

Your answer: **4 个。 $grid_size = \text{ceil}(1024/256) = 4$, 每个 block 输出一个 partial sum。**

Question 3: Why do we use `__syncthreads()` after each reduction step?

Your answer: **因为要保证当前步所有线程写完 shared memory 后, 下一步的读取才开始。若没有同步, 某线程可能读到其他线程还未更新的旧值, 导致结果错误。**

Exercise 2: Analyze Occupancy Results

From the test output, fill in the table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	2	100.0	0.0826
128	4	100.0	0.0629
256	8	100.0	0.0638
512	16	100.0	0.0761
1024	32	100.0	0.0765

Question: Does higher occupancy always mean better performance?

Your answer: 不一定。我在测试所有 *block size* 得到的 *occupancy* 均为 100%，但执行时间差异明显。*block size*=128 最快，说明此时 *block* 数量 (≈ 7813 个) 与 *warp* 效率的平衡最优。*block* 过小 (64) 导致 *global atomic* 次数翻倍增加竞争；*block* 过大 (1024) 则 *tree reduction* 后期大量线程空闲，*warp* 利用率下降。

Exercise 3: Think About Trade-offs

Question 1: With block size 64, how many final atomic operations occur for $N = 1\text{M}$ elements?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 64) = 15625$

Question 2: With block size 1024, how many final atomic operations occur?

Calculation: $\text{grid_size} = \text{ceil}(1,000,000 / 1024) = 977$

Question 3: Why might fewer atomic operations improve performance?

Your answer: 因为所有 *global atomicAdd* 都写同一个地址，必须串行执行。*atomic* 越多，排队等待越长，等同于把并行工作变回串行。减少 *atomic* 次数直接减少了这个串行瓶颈。

Exercise 4: Algorithm Analysis

Question 1: In tree reduction, what fraction of threads are idle after the first step?

Your answer: 第一步后空闲线程比例 = $1/2$ ($\text{stride} = \text{blockDim.x}/2$ ，只有一半线程参与工作，此后每步再减半)

Question 2: Why does warp shuffle not require `__syncthreads()` ?

Your answer: 同一 *warp* 内线程以 *lockstep* 方式执行，天然同步；*shfl_down* 直接交换寄存器，不经 *shared memory*，无需显式同步屏障

Question 3: How does multi-element per thread improve memory bandwidth utilization?

Your answer: 每线程用 *grid-stride loop* 连续读多个元素，内存访问仍 *coalesced*；同时 *block* 数固定为少量 ($\text{CU} \times 4$)，最终 *global atomic* 次数极少，竞争大幅降低

测试用的.hip 文件源码如下:

```

1 #include <hip/hip_runtime.h>
2 #include <stdio.h>
3 #include <fstream>
4 #include <vector>
5
6 __global__ void reduction_tree(const int* input, int* output, int N) {
7     extern __shared__ int sdata[];
8     int tid = threadIdx.x;
9     int idx = blockIdx.x * blockDim.x + threadIdx.x;
10    sdata[tid] = (idx < N) ? input[idx] : 0;
11    __syncthreads();
12    for (int s = blockDim.x/2; s > 0; s >>= 1) {
13        if (tid < s) sdata[tid] += sdata[tid + s];
14        __syncthreads();
15    }
16    if (tid == 0) atomicAdd(output, sdata[0]);
17 }
18
19 int main() {
20     // 随便造点数据
21     int N = 1000000;
22     std::vector<int> h(N, 1);
23     int *d_in, *d_out;
24     hipMalloc(&d_in, N*sizeof(int));
25     hipMalloc(&d_out, sizeof(int));
26     hipMemcpy(d_in, h.data(), N*sizeof(int), hipMemcpyHostToDevice);
27
28     hipDeviceProp_t prop;
29     hipGetDeviceProperties(&prop, 0);
30     int maxWarpsPerCU = prop.maxThreadsPerMultiProcessor / prop.warpSize;
31
32     int block_sizes[] = {64, 128, 256, 512, 1024};
33     printf("%-12s %-12s %-14s %-14s\n",
34           "BlockSize", "Warps/Block", "Occupancy(%)", "Time(ms)");
35     printf("%-12s %-12s %-14s %-14s\n",
36           "_____", "_____", "_____", "_____");
37
38     for (int bs : block_sizes) {
39         int smem = bs * sizeof(int);
40         int grid = (N + bs - 1) / bs;
41
42         // 理论 occupancy
43         int max_blocks_per_cu;
44         hipOccupancyMaxActiveBlocksPerMultiprocessor(
45             &max_blocks_per_cu, reduction_tree, bs, smem);
46         int active_warps = max_blocks_per_cu * (bs / prop.warpSize);
47         float occ = 100.f * active_warps / maxWarpsPerCU;

```

```

48 // 计时
49 hipMemset(d_out, 0, sizeof(int));
50 reduction_tree<<<grid, bs, smem>>>(d_in, d_out, N); // warmup
51 hipDeviceSynchronize();
52
53
54 hipEvent_t s, e; float ms;
55 hipEventCreate(&s); hipEventCreate(&e);
56 hipMemset(d_out, 0, sizeof(int));
57 hipEventRecord(s);
58 reduction_tree<<<grid, bs, smem>>>(d_in, d_out, N);
59 hipEventRecord(e); hipEventSynchronize(e);
60 hipEventElapsedTime(&ms, s, e);
61 hipEventDestroy(s); hipEventDestroy(e);
62
63 printf("%-12d %-12d %-14.1f %-14.4f\n",
64        bs, bs/prop.warpSize, occ, ms);
65 }
66
67 hipFree(d_in); hipFree(d_out);
68 }

```

本实验主要实现了 GPU 并行规约求和,学习了从朴素 atomic 到高度优化的四种方案。Naive Atomic 所有线程争抢同一地址,完全串行化;Tree Reduction 用 shared memory 做二叉树规约,从而可以在 $O(\log N)$ 步完成 block 内求和;Tree + Warp Shuffle 最后一个 warp 改用 `__shfl_down` 寄存器直通,可以省去 `__syncthreads`; Multi-Element Per Thread 每线程先 grid-stride 累加多个元素再树规约,最大化带宽利用并减少 global atomic 次数。

此外,还分析了 Occupancy 与性能的关系,理解了高 occupancy 与高性能之间的联系与差异。

Lab5 : Monte Carlo Integration

Exercise 1: Understand the Algorithm

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer: 因为每个线程通过 `atomicAdd` 各自将自己的贡献累加进同一个 `result` 变量。如果不在线程内提前除以 `n`, 就需要等所有 `y` 值累加完毕后再做一次除法——这需要额外的同步或额外的 `kernel` 启动。在线程内除以 `n`, 每个贡献直接就是最终结果的一部分, 逻辑更简洁, 也避免了大数累加可能带来的溢出。

Question 2: What is the time complexity of the atomic reduction approach?

Your answer: 理论上是 $O(N)$ 。虽然 GPU 有硬件仲裁机制使得 `atomicAdd` 可以分批并发, 但 N 个线程都争抢同一个内存地址, 本质上是串行化的, 最坏情况下每次 `atomic` 操作依次排队, 总时间与 N 成正比。

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer: 先在每个 block 内部声明一个 shared double `sdata[256]`, 让每个线程把自己算出的 `tmp` 写入对应位置并 `syncthreads()` 同步, 然后做 tree reduction, 每轮让前半部分线程把后半部分的值加进来, `s` 从 `blockDim.x/2` 不断右移直到 1, 最终 `sdata[0]` 就是整个 block 的局部和, 再由 thread 0 单独做一次 `atomicAdd(result, sdata[0])`。这样每个 block 只产生 1 次全局 `atomic`, 可以大幅减少竞争。

Exercise 2: Manual Calculation

For Test 1, verify the result manually:

Given: $a = 0$, $b = 2$, $n = 8$ samples

Formula: $\text{result} = \frac{(b-a)}{n} \sum_{i=1}^n y_i = \frac{2}{8} \times \sum y_i$

Your calculation:

- Sum of `y` values: **23.7507**
- Expected result: $(2-0)/8 \times 23.7507 = 5.937675$
- Does it match the program output? **Yes, 程序输出也为 5.937675**

Exercise 3: Scalability Analysis

Record the execution times:

Test	Sample Count	Execution Time
4	100,000	0.093s
7	10,000,000	0.968s
8	100,000,000	8.325s

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer: $0.968 / 0.093 \approx 10.4$ 倍, 远小于样本量增加的 100 倍。

Question 2: Is the scaling linear? Why or why not?

Your answer: 不是严格线性的, 但随 N 增大趋向线性。从 10M 到 100M (10 倍样本), 时间增加 $8.325 / 0.968 \approx 8.6$ 倍, 已接近线性; 而从 100K 到 10M (100 倍样本), 时间只增加约 10 倍, 呈现明显的次线性。原因是在 $N=100K$ 时, `kernel` 启动延迟、内存分配、`Host`→`Device` 数据传输的固定开销占据了总时间的大部分, 实际计算和 `atomic` 的耗时占比很小; 随着 N 增大, 这些固定开销被摊薄, 计算和内存带宽成为主导, 扩展性才趋于线性。

Exercise 4: Bottleneck Analysis

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for $N=1M$ threads?

Hint: Total work = $N \times (\text{compute} + \text{atomic})$

Your calculation: 每个线程总耗时 = 1 (计算) + 100 (atomic 排队), 有效计算时间 = $N \times 1 = 1,000,000$ cycles, atomic 串行耗时 = $N \times 100 = 100,000,000$ cycles, 效率 = 有效计算 / 总时间 = $1 / (1 + 100) \approx 1\%$. 绝大部分时间其实浪费在了 atomic 竞争上。

Question 2: How would shared memory reduction improve this?

Your answer: 每个 block (如256线程) 先在 shared memory 内做 tree reduction, 只需 $\log_2(256)=8$ 步, 仅 1 个 thread 最终做全局 atomicAdd. $N=1M$ 时只有约 3907 次全局 atomic, contention 减少 256 倍。

Question 3: With 256 threads per block and $N=1M$, how many global atomics would shared memory reduction require?

Your calculation: 块数 = $1,000,000 / 256 \approx 3,907$ 块, 全局 atomicAdd 次数为每块1次, 一共3,907 次

Exercise 5: Precision Considerations

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer: 约 15 ~ 16 位有效十进制数字 (符合 IEEE 754 双精度标准, 尾数52位)。

Question 2: If we switch to `float` (32-bit), what problems might occur with $N=100M$ samples?

Your answer: float 只有约 7位有效数字。当累积和很大 (如 $\sim 1e10$), 而每次新增的小值 (如 $\sim 1e2$) 相差超过7个数量级时, 小值会被完全丢弃。对 $N=100M$ 的大规模求和, float 的舍入误差会大幅累积, 导致结果严重不准确。

Exercise 6: Optimization Analysis

Question 1: In the optimized version, how many global atomics occur for $N=1M$ with 256 blocks?

Your answer: 256 次, 每个 block 的 thread 0 做一次 atomicAdd。

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x) ?`

Your answer: 当 N 超过总线程数 $\text{blockDim} \times \text{gridDim}$ 时, 每个线程处理多个元素, 步长为整个 grid 的大小。这样无论 N 多大, 都能用固定数量的 block/thread 处理完, 避免因 N 过大而开太多 block, 同时也让每个线程可以有更多计算量。

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer: tree reduction 后 `sdata[0]` 已经是整个 block 内所有 y 值之和。将 $(b-a)/n_samples$ 乘进去是一个常数缩放, 对最终结果而言等价于对每个元素分别乘。由 thread 0 统一做, 只需 1次乘法而不是 256 次, 既节省计算也减少精度误差。

本实验通过 GPU 实现蒙特卡洛数值积分，综合练习了两种核心 GPU 编程模式，即 Embarrassingly Parallel（各 thread 独立计算积分贡献）和 Reduction（将 N 个贡献值归约为一个结果）。

实验分析了基础版本（atomicAdd）的性能瓶颈，所有线程竞争同一全局地址，使 reduction 实质上串行化，效率约为 1%（compute:atomic=1:100 时），进而引出该问题的优化方案。先在 shared memory 内进行 tree reduction（ $O(\log B)$ 步），再由每 block 的 thread 0 执行一次全局 atomic，将全局 atomic 次数从 N 降至 $N/256$ 。

此外，还讨论了数值精度问题，double 提供约 15-16 位有效数字，大样本下 float 会因精度不足导致结果严重偏差；tree reduction 相比顺序累加在精度上也更优。

Lab6 : K-Means Clustering (Capstone)

Exercise 1: Algorithm Analysis

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer: $O(k)$, 即 $O(100)$ 。每个 thread 负责一个数据点，对所有 k 个质心各计算一次欧氏距离（2次减法、2次乘法、1次加法、1次比较），共 k 轮循环，工作量与 k 成正比。

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer: Global memory 的 atomic 延迟远高于 shared memory，且 N 个线程同时争抢 k 个全局地址会造成严重 contention。改为先在 shared memory 里做 block 内的局部累加（只有 256 个线程竞争 k 个共享地址，速度快），再由每个 block 的 k 个线程将 block 内小计一次性合并到全局（global atomic 次数从 N 次降为 $blocks \times k$ 次），大幅减少全局内存压力。

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer: `finalize_recalculation_and_compare_kernel` 中会检查 `count > 0`：若该质心的点数为 0，则直接保留上一轮的质心坐标（`new = prev`），避免了除以 0 产生 NaN。该质心在下一轮仍会参与竞争，等待点重新分配给它。



Exercise 2: Performance Analysis

Record the execution time for test 4:

- Sample size: 50,000
- Clusters (k): 50
- Max iterations: 100
- Execution time: **0.103s**

Question: What is the dominant computation in each iteration?

Your answer: 每次迭代的主导计算是 Assignment 阶段（`find_nearest_centroids_kernel`）。它需要对每个点计算到所有 k 个质心的距离，共 $N \times k = 50,000 \times 50 = 250$ 万次距离运算，计算量远超 Update 阶段的 N 次 atomic 累加。

Exercise 3: Compute Intensity

For $N = 50,000$ points and $k = 50$ clusters:

Assignment kernel:

- Distance calculations per point: **50 次 (对 $k=50$ 个质心各算一次)**
- Total distance calculations: **$50000 \times 50 = 2500000$ 次**

Update kernel:

- Shared memory atomics: **每个点做 3 次 (累加 x , y , $count$) , 共 $3 \times 50000 = 150000$ 次**
- Global atomics (with 256 threads/block): **每个 block 有 $k=50$ 个线程各做 3 次全局 $atomic$, block 数 $= \lceil 50,000/256 \rceil = 196$, 共 $3 \times 50 \times 196 = 29400$ 次**

Exercise 4: Memory Optimization

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer: 将质心数组预先加载到 *shared* 内存中。每个 *block* 的线程协作地将 k 个质心的 x/y 坐标从 *global memory* 读入 *shared memory*, 然后所有线程从 *shared memory* 读取, 避免了重复的全局内存访问。

Question 2: If $k = 1000$, what would be the impact on cache performance?

Your answer: $k=1000$ 时, 质心数组占用 $1000 \times 2 \times 4 = 8,000$ 字节, 但 *recalculate_centroid_kernel* 中 *shared memory* 数组大小硬编码为 100 (*float sdata[100]*), $k > 100$ 会直接越界溢出, 是当前实现的硬性限制。即便绕过这个问题, *L1 cache* 通常只有 32~64KB, 也很难同时容纳所有质心数据, 每次 *loop* 都会有大量 *cache miss*, 导致 *assignment kernel* 性能急剧下降。



Exercise 5: Convergence Analysis

Question 1: What is the convergence threshold in this implementation?

Your answer: 判断条件为 $dxdx + dydy < 1e-8$, 即质心移动的平方距离小于 $1e-8$, 对应欧氏距离小于 $\sqrt{1e-8} \approx 1e-4$ 个单位时认为该质心已收敛。

Question 2: Why is it important to check convergence rather than always running *max_iterations*?

Your answer: *K-Means* 在实践中往往在远少于 *max_iterations* 的轮次内就已收敛。每次迭代需要三次 *kernel* 启动 + 一次 *Device, Host memcopy* (读取 *num_converged*), 不必要的迭代代价不小。提前终止可以节省大量 *GPU* 时间, 尤其对 *max_iterations* 较大的情况 (如本实验设为 100) 效果显著。

Question 3: How does the double-buffering (swapping *prev* and *new* centroids) help?

Your answer: 每次迭代开头用 *swap(new_centroid_x, prev_centroid_x)* 交换指针, 使上一轮的“新质心”变成本轮的“旧质心”, 而上上轮的“旧质心”缓冲区变成本轮写入新质心的目标, 完全不需要额外的 *memcpy*。这样 $O(1)$ 的指针交换就替代了 $O(k)$ 的数据拷贝, 节省内存带宽并避免分配额外的 *GPU* 缓冲区。

最后的 Lab 6 是综合性 Capstone 实验, 以 K-Means 聚类为载体, 将前几个 lab 的 GPU 编程模式全部集成到一个多 kernel 的迭代 workflows 中, 是本系列实验的一个总结。

实验将算法分解为三个 GPU kernel: 首先 `find_nearest_centroids_kernel` 用 one-thread-per-point 的方式并行执行 $O(N \times k)$ 的距离计算(Embarrassingly Parallel); 然后 `recalculate_centroid_kernel` 采用两级 atomic 策略, 先在 shared memory 做 block 内局部累加, 再合并到 global memory, 大幅降低 contention; 最后 `finalize_recalculation_and_compare_kernel` 完成质心均值计算并检测收敛, 支持提前终止。

除此之外, 实验中还讲解了双缓冲技术(指针交换避免 memcpy)和空簇保护(防止 NaN)。整个流程的复杂度为每迭代 $O(N \times k)$, 其主要瓶颈在 Assignment 阶段。

NIJUB

第二部分 GPU 编程实验

一、实验背景

在前几次的实验中，我们围绕近似最近邻搜索（ANNS）逐步展开优化：从 SIMD 指令级并行，到 Pthread / OpenMP 的多线程并行，再到 MPI 的多进程并行。这些优化从底向上，分别对应指令级、线程级、进程级三个并行层次，但是其本质都是在 CPU 这一类核心数较少、单核性能强的硬件结构上提升算法的并行度。

本次实验将 ANN 搜索进一步迁移至 GPU 上，使用 CUDA 进行编程。GPU 与 CPU 在体系结构上有着本质的差异。GPU 拥有成百上千个轻量级计算核心，采用 SIMT（单指令多线程）的执行模型，以海量线程和高内存带宽支撑其强大的计算能力，尤其适合数据并行、计算密集且访存规整的批量运算。然而，GPU 编程也具有一系列 CPU 上不曾出现的新约束，主机（host）与设备（device）拥有相互独立的内存空间，数据需要显式地在二者之间拷贝传递；核函数（kernel）的启动也存在固定的开销；同时众多线程的访存需要尽量合并（coalesced）才能跑满带宽，否则很容易出现资源的浪费。因此，GPU 编程的优化重心，也从 CPU 并行的如何切分任务这一问题转移到了如何把计算组织成适合 SIMT 的大块规整运算，并尽量减少主机与设备间的数据搬运。

依照实验要求，本次主要在 GPU 上完成以下内容：

- 实现最基础的暴力检索，利用 cuBLAS 将批量距离计算组织为矩阵乘法，并对比 top- k 选取分别放在 CPU 与 GPU 的两种方案；
- 将 IVF 倒排索引迁移到 GPU，把批量查询按不同策略分组、逐簇做矩阵乘精排，并利用簇重叠率等指标分析批量化带来的浪费；同时系统考察 nlist、分组策略、批大小三个方面的影响；
- 进一步探索 IVF-PQ，将之前实验中的全局 PQ 与簇内独立 PQ 两种量化方式进行 GPU 编程的实现，分析其在 GPU 运算环境下的差异与特征。

实验使用 DEEP100K 数据集，在本机 NVIDIA GeForce RTX 4060 Laptop GPU (8GB) 上运行，CUDA Toolkit + cuBLAS 编译。本次实验以单机 CPU (OpenMP) 实现作为各算法的基线来计算加速比。

二、实验原理

(一) GPU 与 CUDA 编程模型

SIMT 执行模型： CUDA 程序由大量线程（thread）组成，线程被组织为线程块（block），若干 block 构成一个网格（grid）。硬件以 32 个线程为一个 warp 作为单位调度，同一 warp 内的线程执行相同指令、处理不同数据，即 SIMT。GPU 通过在大量 warp 之间快速切换来隐藏访存的延迟，因此只有当线程足够多，且访存足够规整时，才能充分利用其算力与带宽，真正发挥 GPU 程序的加速效果。

内存层次与主机/设备的分离： GPU 拥有独立的全局内存（global memory），容量大但延迟高；每个 block 内有低延迟的共享内存（shared memory）可供块内线程协作；此外还有 GPU 自己的寄存器等。而其中关键的是，host（CPU）内存与 device（GPU）内存彼此独立，数据必须经

cudaMemcpy 在 PCIe 总线上显式搬运 (H2D / D2H) 才能实现信息的传递。在实际运行中, 这一搬运的代价往往很高, 是 GPU 程序性能的一个核心瓶颈, 也是本实验中重点探讨的问题之一。

cuBLAS: 对于批量内积这类规整运算, 直接调用高度优化的 cuBLAS 库做矩阵乘 (GEMM) 通常远优于手写 kernel。与普通编程不同的是, cuBLAS 采用列主序, 实现时需对转置与维度做相应换算。

测试说明: 在实验测试方面, 由于 GPU 存在 kernel 启动开销与时钟爬升, 且本机为笔记本 GPU 存在睿频抖动, 本实验在每次运行前都加了一段暖机, 同时重复测量多次取中位数, 以尽可能地减小抖动影响。

(二) 算法回顾

本实验涉及的算法原理均已经在之前的实验中进行了详细的分析与探讨, 此处我们仅做简要回顾。本次实验的内容主要在于如何对其进行 GPU 编程的迁移。

- **暴力检索 (Flat):** 计算查询与全部基向量的距离后取 top- k , 召回率恒为 1。在 GPU 上, M 条查询与 N 条基向量的全部内积可一次性组织为一个 $M \times N$ 的矩阵乘, 由 cuBLAS 高效完成; 随后的 top- k 选取既可以把整个距离矩阵拷回 CPU 再选 (GPU+CPU), 也可以直接在 GPU 上用 kernel 选取 (GPU+GPU)。
- **IVF (倒排文件索引):** 对基向量 KMeans 聚类得 $nlist$ 个簇并按簇内存重排; 查询时先粗排选出最近的 $nprobe$ 个簇, 再精排扫描这些簇内向量。在 GPU 上, 为了让矩阵乘足够大、足够规整, 通常需把一批查询分组 (grouping) 后再逐簇统一进行矩阵乘。而一组查询所涉及的簇集合越分散, 矩阵乘中被真正用到的比例就越低, 为此我们引入了**簇重叠率**指标, 具体见后文的详细描述。
- **IVF-PQ:** 在 IVF 粗筛后, 用乘积量化 PQ 快速估算近似距离, 筛出 $rerank_k$ 个候选后再进行重排。其中分为两种方法, 全局 PQ 全库共享一套量化器、查找表 (LUT) 仅构建一次; 簇内独立 PQ 则为每个簇单独训练量化器, 精度更高但需为每个被探查的簇分别重建 LUT, 且占用的内存会随着簇数而增长。

(三) 批量化与簇重叠率

通过前面的介绍我们知道, GPU 的算力只有在大块规整运算下才能充分发挥。对 IVF 而言, 如果逐条进行查询、逐簇地做小规模矩阵乘, 则会导致每次运算规模过小, 使得 kernel 启动的开销占主导, 效率极低。因此本实验将查询分组处理: 把一组 B 条查询收拢为连续矩阵, 对该组所涉及的每个候选簇 c , 一次性计算“整组查询 $\times$ 簇 c 内全部向量”的矩阵乘。

但是, 这样的方式也带来一个新的问题, 组内不同查询探查的簇各不相同, 记该组所有查询的探查簇的并集为 U 。我们对每个被涉及的簇都让整组查询参与矩阵乘, 而其中只有真正探查该簇的查询的结果是有用的。由此, 可以定义**簇重叠率**:

$$\text{overlap} = \frac{\sum_{\text{query}} nprobe}{B \cdot |U|} \approx \frac{nprobe}{|U|}.$$

overlap 越高, 说明组内查询探查的簇越集中, 矩阵乘中被有效利用的比例就越高, 浪费也就越少。由于一组查询的 $|U|$ 最多为 $nlist$, 因此可以计算出 overlap 的上限约为 $nprobe/nlist$ 。在实现时考虑不同的分组策略, 本质上就是试图通过让探查相似簇集合的查询聚集到同一组内, 从而取得 overlap 的提高。

三、 程序设计与实现

为了全面探讨各个算法的细节与特点，本实验一共实现了 S0~S4 共 5 个策略，算法与其实现方式如下表所示：

表 1: 算法策略

策略编号	算法	说明
0	Flat (GPU+CPU)	cuBLAS 矩阵乘 + 距离矩阵拷回 CPU 选 top- k
1	Flat (GPU+GPU)	cuBLAS 矩阵乘 + GPU 上 kernel 选 top- k
2	IVF	分组方式：顺序 / 按主簇 / 按完整簇集 / query-KMeans
3	IVF-PQ 全局 PQ	全库构建一次，ADC 粗筛 + 重排
4	IVF-PQ 簇内 PQ	每簇独立构建，ADC 粗筛 + 重排

接下来将逐步讲解各个算法 GPU 编程的具体实现。

(一) 通用头文件与共享设备 kernel

由于 GPU 编程需要很多重复的通用逻辑，我们首先实现了两个通用的头文件 common.cuh 与 gpu_common.cuh。其中 common.cuh 主要负责数据读写、计时器、召回率计算、错误检查宏等通用设施，作为封装好的工具以便后续的编程；gpu_common.cuh 则提供了所有策略共享的设备 kernel，包含内积转距离、块级 top- k 、行 gather、PQ 的 LUT 构建与 ADC。

其中较为关键的是块级 top- k 这一部分。每个 block 负责一行（一条查询的全部距离），通过 k 次“块内归约求全局最小值并标记为 INF”在共享内存中选出最小的 k 个。该实现复杂度为 $O(k)$ 次归约，对本实验 $k = 10$ 的场景完全足够。

块级 top- k kernel (gpu_common.cuh, 节选)

```

1 // 每个 block 处理一行(一条 query 的 N 个距离)，选最小的 k 个
2 __global__ void topk_kernel(float* dist, int N, int k, uint32_t* out_idx,
3   float* out_dist){
4   int q = blockIdx.x, tid = threadIdx.x, nt = blockDim.x;
5   float* row = dist + (size_t)q * N;
6   // 共享内存放每个线程的局部最小值与下标
7   for (int it = 0; it < k; ++it){
8       // 1) 每个线程在 strided 范围内求局部最小
9       // 2) 块内树形归约得到本行全局最小，写出第 it 个结果
10      // 3) 将该位置置 +inf，进入下一轮
11  }

```

此外，ip_to_dist 把内积矩阵逐元素转为 $1 - \langle q, x \rangle$ ；gather_rows 按索引把一组查询收拢为连续矩阵；build_lut 与 adc_kernel 则服务于 IVF-PQ。

(二) 暴力检索 (gpu_flat.cuh)

暴力检索使用一个持久化上下文 FlatCtx, 将 base、query 一次性上传、显存与 cuBLAS 句柄一次性准备 (不计入查询耗时); 查询时只做 GEMM 与 top- k 。两种方案的唯一区别在于 top- k 的位置, GPU+CPU 把整个 $M \times N$ 距离矩阵拷回 host 再用 OpenMP 选取, GPU+GPU 则直接在设备上用 topk_kernel 选取、只把 $M \times k$ 个下标拷回。

Algorithm 1 Flat 批量检索 (GPU+GPU, 策略 1)

Input: $\text{base} \in \mathbb{R}^{N \times d}$ 、 $\text{query} \in \mathbb{R}^{M \times d}$ 、 k

Output: 每条查询的 top- k

- 1: ▷ 一次性: 上传 base/query, 分配 dS, 建 cuBLAS 句柄
 - 2: $dS \leftarrow \text{cuBLAS-GEMM}(\text{query} \times \text{base}^\top)$ ▷ $M \times N$ 内积矩阵 (列主序换算)
 - 3: $dS \leftarrow 1 - dS$ ▷ 内积转距离
 - 4: $(idx, dist) \leftarrow \text{topk_kernel}(dS, N, k)$ ▷ GPU 上逐行选 top- k
 - 5: 把 idx (仅 $M \times k$ 个) 拷回 host ▷ D2H 极小
-

(三) IVF (gpu_ivf.cuh)

IVF 的离线部分 (KMeans 聚类、按簇重排) 仅需一次, 在 CPU 上完成即可。在线查询则在 GPU 上分为粗排、分组、逐簇精排与合并这三个阶段。粗排用一次 $\text{query} \times \text{centroids}^\top$ 矩阵乘加 top- k 选出每条查询的 nprobe 个簇; 分组按 group_mode 把查询划入若干组; 精排则对每组、每个被触及的簇做整组 $\times$ 簇内向量的矩阵乘, 再逐查询合并其真正探查的簇的结果。

Algorithm 2 IVF 批量分组检索

Input: query、nprobe、group_B、group_mode、 k

Output: 每条查询的 top- k

- 1: $probe \leftarrow \text{coarse}(\text{query}, \text{nprobe})$ ▷ 粗排: 每条查询的 nprobe 个候选簇
 - 2: $groups \leftarrow \text{make_groups}(\text{group_mode}, probe, \text{query}, \text{group_B})$ ▷ 分组
 - 3: **for** 每个 group g **do**
 - 4: $U \leftarrow g$ 内所有查询探查簇之并集; 记录每簇被组内哪些查询命中
 - 5: **for** $c \in U$ **do**
 - 6: $S \leftarrow \text{cuBLAS-GEMM}(g \times \text{cluster}_c^\top)$; $S \leftarrow 1 - S$
 - 7: topk_kernel(S), 把命中该簇的查询的候选并入各自的堆
 - 8: **end for**
 - 9: **end for**
 - 10: **return** 各查询堆中的 top- k (局部下标经重排表还原为全局 id)
-

四种分组策略 (make_groups) 分别如下:

- **顺序分组:** 按查询顺序切块, 作为基线;
- **按主簇分组:** 按每条查询最近的那个簇排序后切块;
- **按完整探查簇集合分组:** 把每条查询的 nprobe 个簇排序成签名、按签名字典序排序后切块, 是唯一直接压缩组内 $|U|$ 的策略;
- **query-KMeans 分组:** 对查询向量做 KMeans, 把相似查询聚到一组。

其中按完整探查簇集合分组与 query-KMeans 分组主要为了提高簇重叠率, query-KMeans 则有额外引入 KMeans 的分组开销。

(四) IVF-PQ: (gpu_ivf_pq.cuh)

与之前一样，本次实验实现两种 IVF-PQ: **全局 PQ** 全库共享一套码本，LUT 每组只构建一次；**簇内 PQ** 为每个簇用其簇内向量独立训练一套码本（簇间并行），LUT 需在进入每个候选簇前用该簇码本重建。两者都在粗筛出 `rerank_k` 个候选后再进行重排得到最终的 `top-k`。

Algorithm 3 IVF-PQ 精排（簇内 PQ + 重排，策略 4）

Input: 一组查询 g 、候选簇集合 U 、每簇码本、`rerank_k`、 k

Output: 每条查询的 `top-k`

```

1: for  $c \in U$  do
2:    $LUT \leftarrow \text{build\_lut}(g, \text{码本}_c)$  ▷ 簇内 PQ: 用本簇码本重建 LUT
3:    $S \leftarrow \text{adc\_kernel}(\text{code}_c, LUT)$  ▷ 查表得近似距离
4:    $\text{topk\_kernel}(S, \text{rerank\_k})$ , 把候选并入各查询的候选堆（按近似距离）
5: end for
6: 重排: 对每条查询的 rerank_k 个候选，用全精度向量算精确内积，取最终 top-k

```

(五) 批量测试框架

与前几次实验一致，由于策略与参数众多，且笔记本 GPU 存在睿频抖动，本实验也在 AI 的协助下，改写实现了一版命令行参数化的主程序 `d_main.cu` 以及对应的用于批量测试的脚本 `run_batch.py`（详见代码仓库）。脚本定义了测试矩阵，对每个配置用 `subprocess` 独立运行一次完整的进程，每次独立建索引、独立初始化 CUDA 上下文，最后抓取其 `stdout` 的结果写入 `result.csv`。每个配置在进程内重复 5 次取中位数，索引构建等一次性开销不计入查询耗时。

四、实验结果分析

(一) 数据总览

本实验在 DEEP100K 上对全部策略进行测试（2000 次查询， $k = 10$ ，`nprobe`=16）。表 2 汇总了各策略在代表性配置下的结果，加速比相对各算法对应的 **CPU（OpenMP）实现** 作为基线得出。

表 2: 实验数据总览

策略	算法 / 配置	召回率	延迟 ($\mu\text{s}/\text{query}$)	加速比 (vs CPU)
0	Flat GPU+CPU	1.0000	127.6	3.55×
1	Flat GPU+GPU	1.0000	25.1	17.48×
2	IVF <code>nlist</code> =32 (<code>probeset</code> , $B=1024$)	0.9965	13.9	18.33×
2	IVF <code>nlist</code> =64 (<code>probeset</code> , $B=1024$)	0.9910	16.8	7.39×
2	IVF <code>nlist</code> =256 (<code>seq</code> , $B=1024$)	0.9583	46.4	0.75×
3	IVF-PQ 全局 (<code>PQ_M</code> =16, <code>rr</code> =200)	0.9681	206.1	0.61×
4	IVF-PQ 簇内 (<code>PQ_M</code> =16, <code>rr</code> =200)	0.9901	216.2	0.59×

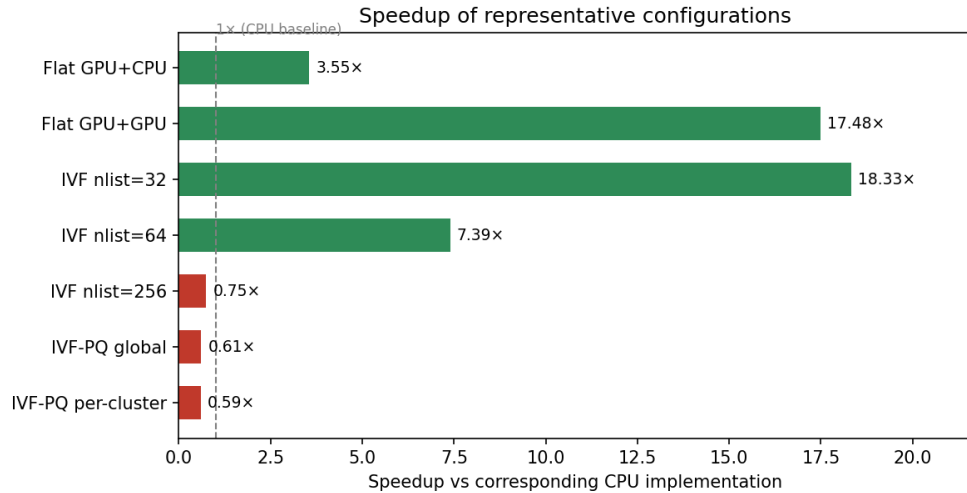


图 1: 各策略代表性配置加速比对比

总体来看, 暴力 GPU+GPU 在精确检索的场景下已经具有极大的效率优势, 延迟仅 $25.1 \mu\text{s}/\text{query}$, 相对 CPU 暴力的加速比高达 $17.5\times$, 体现出 GPU 的强大算力。IVF 的表现则高度依赖 `nlist`, 在小 `nlist` 下加速比可达近 $20\times$, 但随 `nlist` 增大迅速下滑, 甚至慢于 CPU, 但在 `nlist=32`, `probeset`, `B=1024` 的配置下, 其 $13.9 \mu\text{s}/\text{query}$ 的延迟与 0.9965 的高召回也能体现出 IVF 算法在 GPU 环境下的巨大潜力。IVF-PQ 则始终效果较差, 它在本实验 GPU 上几乎无法提供任何速度收益, 但从内存角度来看, 其价值主要在于内存压缩及其与精度的权衡, 也在特殊环境下具有一定的优化意义。

下面将对这几种算法的测试结果进行详细分析。

(二) 暴力检索: GPU+CPU 与 GPU+GPU

表 3 记录了两种暴力算法在不同批大小 M 下的每查询平均延迟, 以及相对于 CPU 暴力基线的差异。

表 3: 暴力检索随批大小 M 的平均延迟 ($\mu\text{s}/\text{query}$), 召回率均为 1

M	GPU+CPU	GPU+GPU	CPU 暴力	GPU+GPU 加速比
64	185.8	11.2	517.2	$46.3\times$
256	153.2	26.5	479.9	$18.1\times$
512	151.4	24.0	438.2	$18.2\times$
1024	137.9	25.3	442.8	$17.5\times$
2000	127.6	25.1	438.9	$17.5\times$

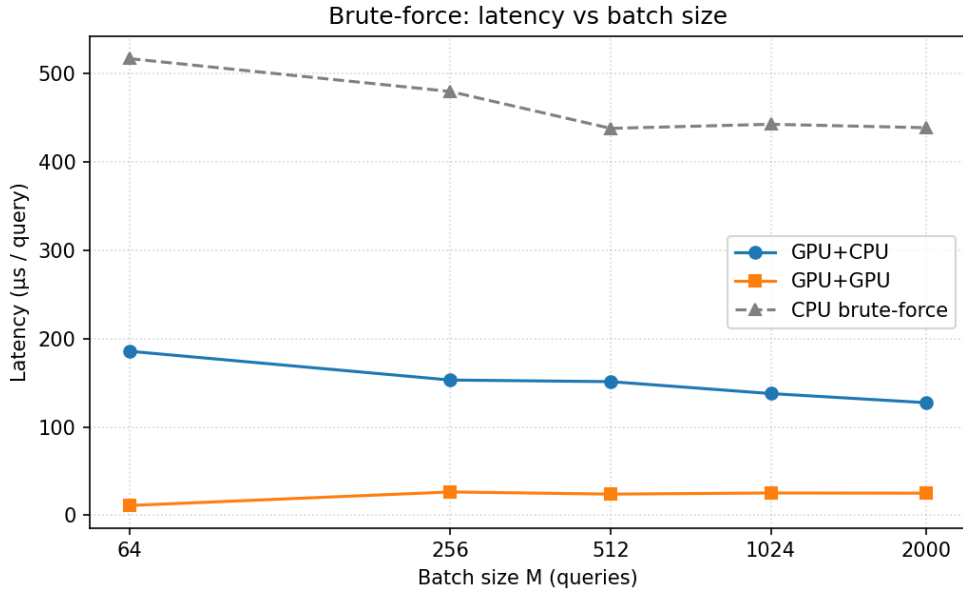


图 2: 暴力检索两种 top-k 方案的批量扩展性

数据呈现出显著的规律，GPU+GPU 全程显著快于 GPU+CPU (25.1 vs 127.6 $\mu\text{s}/\text{query}$ ，约 5 倍差距)。从查询请求来看，二者的矩阵乘部分应该是完全相同的，因此其性能差异完全来自于 top-k 位置的不同。GPU+CPU 必须把整个 $M \times N$ 的距离矩阵复制回主机 ($M=2000$ 时即 $2000 \times 10^5 \times 4 \approx 800 \text{ MB}$ 的 D2H 传输)，这一数据搬运带来的巨大开销最终成为了该算法的绝对瓶颈，随着查询数量的持续增大，甚至还有可能导致性能的崩溃；而 GPU+GPU 则是选择了把归约留在设备上，只回传 $M \times k$ 个下标，这一开销极小，几乎可以忽略。这也恰恰印证了 GPU 编程的核心原则，也就是尽量减少主机与设备间的数据搬运，把归约留在设备侧来实现。

此外，也可以看到 CPU 暴力算法每个查询的平均延迟恒定在约 440–520 μs (与 M 无关，符合其逐条线性扫描的特征)，而 GPU+GPU 在 $M \geq 256$ 后稳定在约 25 μs ，呈现出良好的线性扩展。这里 $M=64$ 一行 (11.2 $\mu\text{s}/\text{query}$ 、46 $\times$) 为亚毫秒级的小批量测量，其延迟受 kernel 启动与时钟爬升影响抖动较大，真正可靠的结果主要来自 $M \geq 256$ 的测试。

(三) IVF 相关参数分析

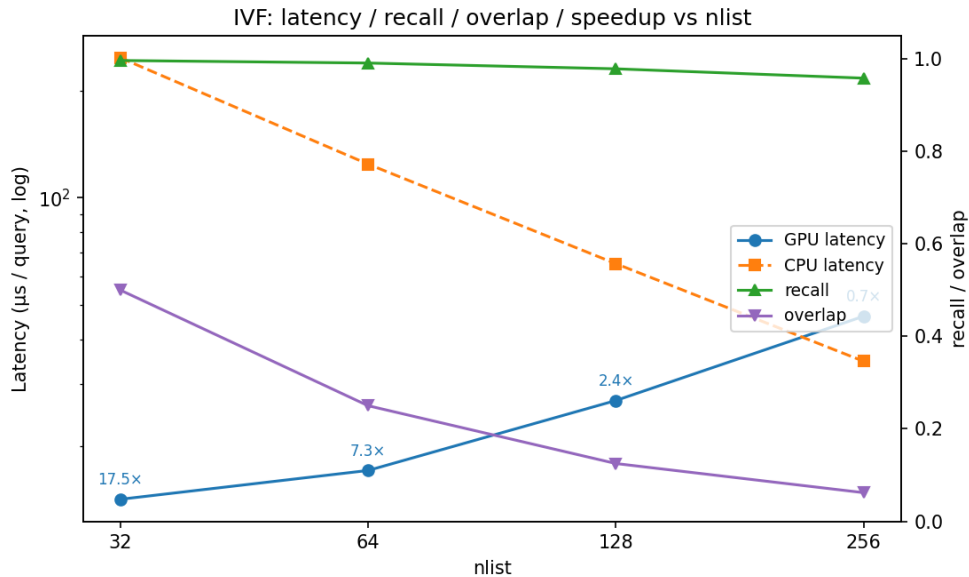
1. nlist 对比

表 4 固定分组方式为顺序分组，批大小 $\text{group_B}=1024$ ，记录了不同 nlist 的测试结果。从中可以看出，nlist 在 IVF 算法中具有决定性的影响。

表 4: IVF 随 $nlist$ 的变化 ($nprobe=16$, 顺序分组, $B=1024$)

$nlist$	召回率	overlap	GPU($\mu s/q$)	CPU($\mu s/q$)	加速比
32	0.9965	0.500	14.2	247.7	17.46$\times$
64	0.9910	0.250	17.1	124.4	7.28 $\times$
128	0.9785	0.125	26.9	65.3	2.43 $\times$
256	0.9583	0.063	46.4	34.7	0.75$\times$

随着 $nlist$ 从 32 增至 256, 四项指标呈现出的变化趋势高度一致, 召回率从 0.9965 降至 0.9583; overlap 精确地等于 $nprobe/nlist$ (0.500/0.250/0.125/0.063); CPU 延迟则逐步下降 (簇变多的同时也变小了, 每条查询实际扫描的点数 $\approx nprobe \cdot N/nlist$ 随 $nlist$ 反比而减小); 而 GPU 延迟反而上升。两者一升一降, 使加速比从 17.5 $\times$ 一路降低至到 0.75 $\times$, 以致在 $nlist=256$ 时出现了负优化的结果。

图 3: IVF 关键指标随 $nlist$ 的变化

其根源主要在于 GPU 与 CPU 对簇大小的偏好截然相反, $nlist$ 越小簇越大, 逐簇矩阵乘规模也就越大越稠密, kernel 启动次数减少, 对于 GPU 来说额外的开销也就越少, 从而性能更优; 而当 $nlist$ 变大时, 程序退化为大量微小矩阵乘, 这意味着大量的 kernel 启动与频繁的逐簇结果回传, 使得 GPU 效率骤降。与此同时, overlap 也随 $nlist$ 增大而降低 ($|U|$ 趋近 $nlist$), 批量矩阵乘中被浪费的比例也越来越大。

因此, $nlist$ 这一参数对于 GPU-IVF 具有决定性的作用。从测试结果来看, 适当偏小的 $nlist$ 既能让 GPU 高效地运行, 又能换来更高的召回率, 虽然扫描了更多的点, 但是 GPU 算力充裕, 这一微小的代价本就不是很显著; 而相比于较大 $nlist$ 下大量的 kernel 启动来说, 其整体开销显然是更优的。

2. 分组策略对比

表 5 固定 $nlist=64$ ，对比了四种分组策略在不同批大小下的平均延迟与 overlap。

表 5: 四种分组策略对比 ($nlist=64$, 延迟 $\mu s/q$ & overlap), 召回率均为 0.9910

分组策略	$B=256$	$B=512$	$B=1024$
顺序	32.0 / 0.250	21.7 / 0.250	17.1 / 0.250
按主簇	31.6 / 0.251	21.6 / 0.250	16.9 / 0.250
按完整簇集	30.3 / 0.260	21.4 / 0.259	16.8 / 0.256
query-KMeans	40.4 / 0.256	30.2 / 0.253	20.7 / 0.250

从测试结果来看，四种分组下召回率完全一致（均为 0.9910），与我们的预期一致。在算法中，分组策略只改变了哪些查询一起算，并不会对每条查询实际探查的簇产生影响，因此差异只体现在 overlap 与延迟上。

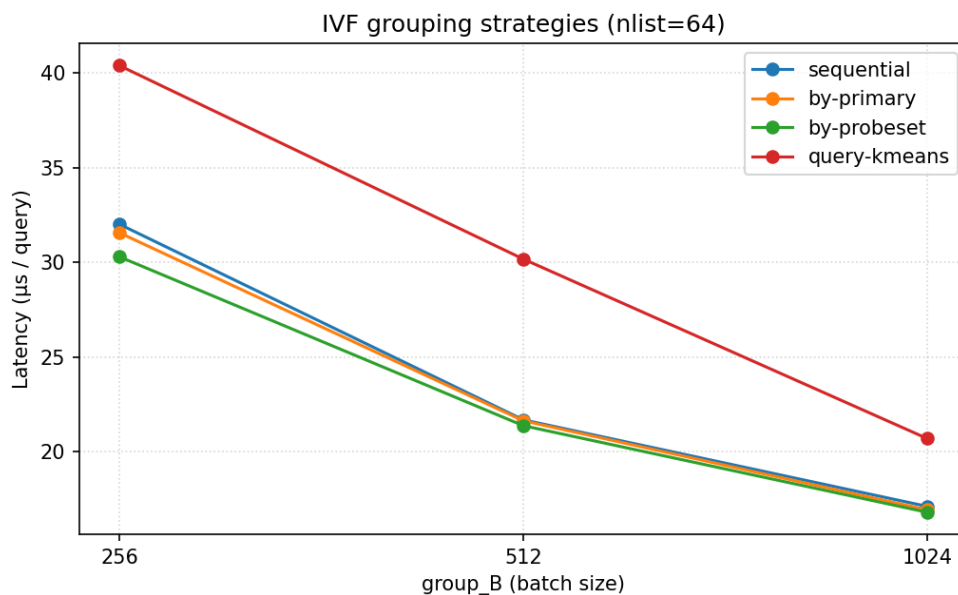


图 4: 四种分组的延迟随 group_B 的变化

这四种分组策略的 overlap 均位于 $nprobe/nlist = 0.25$ 附近，即使一组只有 128 条查询，它们合起来也几乎触及了全部 64 个簇 ($|U| \rightarrow nlist$)，分组难以对于压低 $|U|$ 产生有效的影响。这也说明在本数据规模下，分组提高重叠率的优化空间相当有限。

其中按完整簇集分组略有收益，这一分组方式用整套探查簇集合做键，是唯一真正去压缩 $|U|$ 的策略，overlap 升至 0.256~0.260，并在每个批大小下都取得最低延迟，虽然有一定的效果，但其优势实在较小。query-KMeans 分组虽然在 overlap 也有一定的微小提高，但其算法本身的开销却导致了查询耗时的增大 ($B=512$ 时 30.2 vs 顺序 21.7 $\mu s/q$)，从结果来看反而得不偿失了。

此外，同一策略下， B 从 256 增至 1024，而平均延迟近乎减半（顺序 32.0 $\rightarrow$ 17.1 $\mu s/q$ ），这体现出更大的组带来更大的矩阵乘，kernel 启动的减少使得整体的平均延迟反而大大降低了。

由上述分析可以得出，在 GPU IVF 上，真正决定性能的依然是 `nlist` 与批大小，考虑分组策略的差异只能作为次要的微调；其中按主簇 / 按完整簇集这类较为简单的排序分组实际是最划算的，而 query-KMeans 为代表的相对复杂的分组策略反而得不偿失。

(四) IVF-PQ

表 6 为固定 `nlist`= 64、按主簇分组、 $B=512$ ，对比全局 PQ 与簇内 PQ 在不同 `PQ_M` 与 `rerank_k` 下的召回率与平均延迟。

表 6: 全局 PQ (策略 3) 与簇内 PQ (策略 4) 对比

PQ_M	rerank_k	全局 PQ 召回	簇内 PQ 召回	平均延迟 S3 / S4 ($\mu\text{s}/\text{q}$)
4	50	0.314	0.680	60.9 / 68.2
4	100	0.454	0.808	107.8 / 115.4
4	200	0.612	0.899	207.9 / 209.8
8	50	0.547	0.818	60.4 / 65.7
8	100	0.696	0.914	107.5 / 113.3
8	200	0.822	0.965	201.9 / 208.2
16	50	0.810	0.946	64.3 / 73.2
16	100	0.915	0.982	111.9 / 120.6
16	200	0.968	0.990	206.2 / 216.2

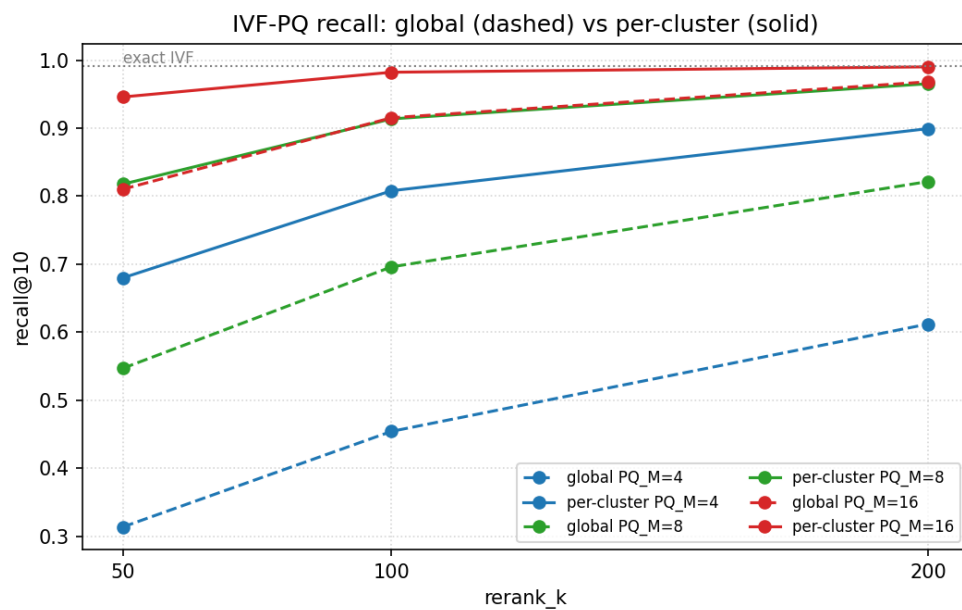


图 5: 全局 PQ 与簇内 PQ 的精度对比

观察数据中可以看出，召回率随 `PQ_M` 与 `rerank_k` 单调上升，段数越多量化越细，重排候

选越多则越接近精确。这些特征与我们之前在 CPU 上的实验结果基本一致。

簇内 PQ 在 $PQ_M=16$ 、 $rerank_k=200$ 时达 0.990，已逼近精确的 IVF。而从整体来看，簇内 PQ 在每个配置下也均显著优于全局 PQ，其原因也与之之前实验所分析的相同，主要在于每个簇的量化器只需拟合该簇内那一小部分分布紧凑的向量，量化误差远小于用同一套量化器去覆盖整个空间；其代价则是码本的内存会随簇数增长而增大（全局 PQ 压缩约 43 $\times$ ，簇内 PQ 因需存 $nlist$ 套码本，压缩降至约 5.4 $\times$ ），且需逐簇重新独立构建。

观察数据趋势，其延迟主要由 $rerank_k$ 决定（ $rr=200$ 时高达 200+ $\mu s/q$ ），这是因为重排阶段的块级 top- k 复杂度随候选数线性增长；而 PQ_M 对延迟的影响则相对较小。

此外，值得强调的是，IVF-PQ 在本实验的 GPU 上并没有带来速度的明显提升。精确的 GPU-IVF ($nlist=64$) 仅需约 17~32 $\mu s/query$ 且召回 0.991，而 IVF-PQ 各配置均在约 60~216 $\mu s/query$ 。这是因为在 GPU 上 cuBLAS 的精确距离计算已经极快，而 PQ 路径额外引入了 ADC、随 $rerank_k$ 增长的 top- k 以及重排等一系列的开销，反而拖慢了整体的延迟。但是从其他角度来看，PQ 的价值其实并不在于速度，而在内存压缩这一方面，考虑一些特定的场景，例如严格限制内存而精度要求适中的场景下，这一算法也可以为我们提供一种有效的思路。

(五) 参数权衡

综合各项数据，我们也可以总结出本实验有关 GPU 的参数调优方案。其最主要的调节手段在于 $nlist$ ，较小时可以同时带来高召回与 GPU 友好的大矩阵乘，从而带来加速比的提升；较大的 $nlist$ 则完全相反。如果追求 GPU 上的高吞吐，优先应当选取适当偏小的 $nlist$ 。

批大小也对于 GPU 利用率具有很大的影响，无论暴力还是 IVF，增大批量都能摊薄 kernel 启动与固定开销，同时增大矩阵乘规模，从而提升单位查询效率。因此我们的 GPU 程序应当尽量地批量化。与此同时，数据搬运也是一种隐形的性能瓶颈，暴力 GPU+CPU 的性能劣势、IVF 大 $nlist$ 下频繁的逐簇回传，也都指向了同一规律，即应当尽量把计算与归约留在设备侧，从而尽可能地减少主机与设备间的通信开销。

本实验涉及的这几种参数效果均很直观，且各个参数带来影响大小的差异也十分显著，因而在调参时无需考虑过于复杂的参数耦合。遵循上述规则，我们可以很轻易地找到该数据规模下最优的参数策略。

此外，IVF-PQ 的两种方法也与之之前实验的结论完全一致，我们不再重复。这里的 PQ 主要适用于库极大、显存无法容纳全精度向量、且对于精度要求较低的场景；在本实验规模下，精确的 IVF 算法显然全面占优。

(六) 问题探讨

除了上述分析之外，实验过程中还有一些问题值得深入探讨。

1. 为什么分组策略收益有限？

因为 overlap 的上限是 $nprobe/nlist$ ，而组内查询的探查簇并集 $|U|$ 很容易触及 $nlist$ ，对于分组策略的调整很难再压低它。只有按完整探查簇集合排序可以把相同 / 相近簇集的查询聚拢，从而略微提高 overlap；而 query-KMeans 的分组开销过大，即使 overlap 有所提升，最终的平均延迟反而却是升高的，也难以带来实际的正向收益。这其实说明并非所有直觉上更优的预处理都能取得优化效果，在 GPU 上，简单廉价的排序分组配合大批量，才是性价比最高的组合。这其实也符合 GPU 这一硬件最初的设计理念。

2. 为什么 PQ 在 GPU 上没有提速？

在 CPU 的架构中, PQ 的 ADC 查表加和远快于逐维浮点距离, 故而可以大幅提升速度; 但在 GPU 上, cuBLAS 的精确矩阵乘已经极快, PQ 路径反而额外背上了 ADC、随候选数增长的 top- k 、逐簇 LUT 构建与重排等开销, 最终导致整体效果更慢。

这也提醒我们, 在 CPU 上有效的优化手段, 迁移到 GPU 后未必成立, 以致出现负优化的结果也不是没有可能。在实验中, 我们必须要结合 GPU 的算力、带宽与启动开销等特征重新进行评估, 依照 GPU 的硬件结构来进行相关优化算法的设计与实现。

五、 总结与体会

本次实验将 ANN 搜索从 CPU 迁移到 GPU, 使用 CUDA 与 cuBLAS 实现并系统对比了暴力检索、IVF、IVF-PQ 一共五类策略, 并通过统一的计时与召回检验, 使用命令行参数化的主程序配合批量测试脚本完成了大规模的参数扫描与分析。

从结果来看, GPU 并行的收益高度依赖于运算能否组织成大块规整的矩阵乘, 以及数据搬运能否被压到最低。暴力 GPU+GPU 以一次大矩阵乘加设备侧 top- k 取得了 $17.5\times$ 的加速, 而 GPU+CPU 仅因把距离矩阵传回主机就慢了约 5 倍; IVF 在小 `nlist` 下加速近 $20\times$, 却在在大 `nlist` 下因簇过小、批量浪费而退化至负优化。这些让我深刻体会到, GPU 编程的性能往往不取决于算得快不快, 而是取决于有没有提供足够大且足够规整的具体计算任务, 以及对于数据搬运这一固定开销的处理方式。

另一个体会则来自 IVF-PQ。它在 CPU 时代是以查表加速距离计算的经典手段, 但迁移到 GPU 后, 面对已经极快的 cuBLAS, 其 ADC 与重排开销反而成了负担、无法提速。这组对照让我深刻认识到, 并行优化必须结合具体硬件的特征重新审视分析, CPU 上的优化算法在 GPU 上未必成立, 反之亦然。而簇内 PQ 全面优于全局 PQ 的结果, 也再次印证了紧凑局部分布这一提升精度的通用思路。

结合前几次实验, 我对于指令级、线程级、进程级再到数据并行 (GPU) 这一完整的并行层次有了更加系统的认识, 不同的体系结构有着不同的代价模型与最佳实践, 只有真正理解硬件, 真正理解算法的计算、访存、通信特征, 才能在具体场景中选出真正有效的并行策略。这也是本次 GPU 编程实验, 乃至本学期的《并行程序设计》这门课程为我带来的最宝贵的收获。

六、 附录: 代码仓库链接

GitHub: <https://github.com/Iriasoul/Parallel-Programming/tree/main/lab05>

七、 附录: AI 使用说明

与之前的实验相比, 此次实验代码量相对较少, 且逻辑与之前基本一致, 因此此次的批量测试框架可以直接在上次的由 AI 辅助下完成的框架上进行删改即可使用; 以及数据可视化部分与之前一样, 由 AI 辅助完成了 Python 绘图相关的编码工作。

此外, 由于第一次使用 GPU 编程与 CUDA, 本次主要在环境相关的一些问题上询问了 AI, 具体对话可直接访问对话导出的链接: (使用的大模型为 Gemini 3.1Pro)

<https://gemini.google.com/share/c8e3a8bf8b08>

END